



Welcome to

MoonGen BlueField-2 Packet Generation

BornHack July 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, <https://codeberg.org/kramse/dpu-experiment-2026.tex> in the repo security-courses

Experiment Overview



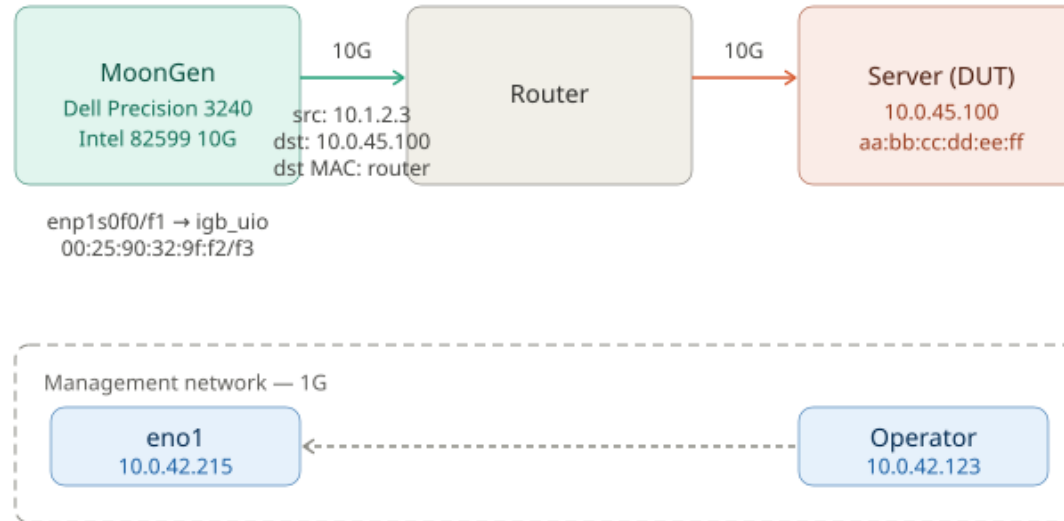
Goal: reproduce and extend the 2025 version of the DPDK/MoonGen packet generation results on hosts T150 and basse equipped with an identical BlueField-2 DPU network cards

Both hosts use the penguinping MoonGen scripts against a TCP SYN target, transmitting via the BF2's integrated ConnectX-6 Dx controller

Source for the tools:

- <https://github.com/tumi8/MoonGen.git> TUM School of Computation, Information and Technology Technical University of Munich
- <https://codeberg.org/kramse/penguinping> My Lua scripting to simulate hping3 using Lua
- <https://penguinping.org> Home page for the tool Lua scripting to simulate hping3 using Lua – to be updated!!!!111

Penguinping testing



Penguinping Software stack



I have previously used a nice library Libmoon and MoonGen for creating packets with Lua and 10G cards – one CPU core could push 14million packets per second

Original paper: P. Emmerich et al., *MoonGen: A Scriptable High-Speed Packet Generator*, ACM IMC 2015.
<https://dl.acm.org/doi/10.1145/2815675.2815692>

Updated fork with MoonEm path emulator (delay, rate limiting, packet loss): TU Munich Network Architectures and Services group, CoNEXT 2025. <https://github.com/tumi8/moongen>

libmoon (core DPDK/LuaJIT framework underlying MoonGen): <https://github.com/libmoon/libmoon>

This is confirmed working on Debian 13 in 2026!

It used DPDK, Just-in-time compilation and is very nice

I promise to update <https://penguinping.org/> and <https://codeberg.org/kramse/penguinping>

PenguinPing – re-implementing hping3 with Lua



```
root@penguin01:~/projects/MoonGen# ./build/MoonGen ./examples/penguinping-02.lua 10.0.49.1 -a 10.1.2.3 -r 10000 -S -p 80
[INFO]   Initializing DPDK. This will take a few seconds...
[INFO]   Found 2 usable devices:
        Device 0: 00:25:90:32:9F:F2 (Intel Corporation 82599ES 10-Gigabit SFI/SFP+ Network Connection)
[INFO]   Device 0 (00:25:90:32:9F:F2) is up: 10000 MBit/s
TCP mode get TCP packet
IP4 10.1.2.3 > 10.0.49.1 ver 4 ihl 5 tos 0 len 46 id 0 flags 0 frag 0 ttl 64 proto 0x06 (TCP) cksum 0x0000 [-]
TCP 52049 > 80 seq# 1 ack# 0 offset 0x5 reserved 0x00 flags 0x02 [-|-|-|SYN|-] win 10 cksum 0x0000 urg 0 []
0x0000:  0000 0000 0000 0025 9032 9ff2 0800 4500
0x0010:  002e 0000 0000 4006 0000 0a01 0203 0a00
0x0020:  3101 cb51 0050 0000 0001 0000 0000 5002
0x0030:  000a 0000 0000 0000 0000 0000

[Device: id=0] TX: 14.88 Mpps, 7619 Mbit/s (9999 Mbit/s with framing)
[Device: id=0] TX: 14.78 Mpps, 7568 Mbit/s (9933 Mbit/s with framing)
[Device: id=0] TX: 14.88 Mpps, 7619 Mbit/s (10000 Mbit/s with framing)
```

- Using Lua we can implement the same attacks from Hping3 easily
- Only about 230 lines of Lua using MoonGen and libmoon
- Can run at specific rate up to full 10Gbps / 14.8 Million packets per second using a single CPU core

Comparable to real DDoS?



Tools are simple and widely available but are they actually producing same result as high-powered and advanced criminal botnets. We can confirm that the attack delivered in this test is, in fact, producing the traffic patterns very close to criminal attacks in real-life scenarios.

- We can also monitor logs when running a single test-case
- Gain knowledge about supporting infrastructure
- Can your syslog infrastructure handle 800.000 events in < 1 hour?

Main difference are that attackers are free to switch attack types and mix them. While we try specifically to keep using one type, to see the worst and which ones that hurt the most.

I also start at the bottom, and work my way up – while an attacker may begin attacking HTTP/HTTPS directly.

Nvidia Mellanox DPU2



MT42822 — ConnectX-6 Dx network controller PCIe Gen4 host interface

2× 100G ports (SFP28/DAC)

8× Arm Cortex-A72 onboard cores – not used in this experiments, slower than main CPU

Drivers: mlx5_core (host), mlx5 PMD (DPDK)

Hardware: Small Dell PC architecture



- Dell PowerEdge T150: Intel Xeon E-2314, 4 cores (AVX-512 capable)
Debian 13 (trixie), kernel 6.12.90+deb13.1-amd64 BlueField-2 DPU, MT42822, integrated ConnectX-6 Dx, dual 100G ports
- Dell Precision 3640 Basse: Intel Core i9-10900K, 10 physical cores / 20 threads (no AVX-512)
Debian 13 (trixie), kernel 6.12.95+deb13-amd64 BlueField-2 DPU, same model as T150
- Port 0 on both systems connected to a Mellanox SN2700 switch

Software Stack



- NVIDIA DOCA-Host 3.4.0 (`doca-host_3.4.0-085000-26.04-debian13_amd64.deb`) on both hosts
- DPDK via `dpdk-community` (DOCA-bundled), MoonGen built from a fresh clone against each host's vendored DPDK
- MLNX-OFED DKMS modules (`mlnx-ofed-kernel`, `iser`, `isert`, `srp`, `xpmem`) installed and matching between hosts
- A bit of fiddling with kernel options `/etc/default/grub/`:
`GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=off default_hugepagesz=2MB hugepagesz=2MB huge`

Initial Results



T150 Results: Verified

Queues	3 TX queues, 3 of 4 CPU cores
Peak throughput	approximately 43 Mpps
Line rate	approximately 20 Gbps
Frame size	60 bytes
Rate limiting	hardware setRate API unsupported on BF2, silently ignored

Basse Results: Verified

Queues	9 TX queues, 9 of 10 physical cores
Peak throughput	approximately 97 to 98 Mpps sustained
Verification	hardware phy counters (host) and switch port counters (SN2700) both confirm real traffic

Combined traffic would be close to the maximum for a 100Gbps port with small packets 140Mpps

Packet Generation Summary — MoonGen on BlueField-2



Hosts: Dell PowerEdge T150, Intel Xeon E-2314 (4 cores, 2.80 GHz)

Dell Precision 3640 Intel Core i9-10900K, 10 physical cores / 20 threads (no AVX-512)

NIC: NVIDIA BlueField-2, single 100G port

Software: MoonGen + libmoon updated by TUM School of Computation, Information and Technology Technical University of Munich , DPDK m1x5 PMD

Packet size: 60 bytes (minimum Ethernet frame)

TX queues: 3 (1 core reserved for DPDK master)

Result: ~43 Mpps / ~20 Gbps

Verified via ethtool -S hardware counters (rx_packets_phy)

Bottleneck: CPU cores — scales linearly with queue count

It's in my tent on PTY, if you want to try it! aaaand I brought a 10-core CPU PC

Context: ConnectX-6 Dx Ceiling



Independent DPDK community benchmarks on the same ConnectX-6 Dx silicon report a single-port ceiling of approximately 148 Mpps at 64-byte frames, and approximately 210 to 214 Mpps combined when both ports of a dual-port card are driven simultaneously

Basse's verified approximately 98 Mpps single-port result sits at roughly two thirds of that single-port ceiling, with core count (9 of 10 physical cores already committed to TX) as the likely next constraint rather than the NIC itself

Next Steps:

- Push queue count and burst/cache tuning further on basse to probe the single-port ceiling
- Connect basse's second BF2 port for genuine dual-port testing, budgeting CPU cores accordingly
- Make more Lua scripting to expand scenarios for testing – what about 100Mpps of QUIC?

Questions?



Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hlk@zencurity.com